

《 Introduction to Embedded System 》

Lab-2

Major: **Software Engineering**

Name: **Md Tasnimul Hasan**

Student ID: **202480090104**

Task 1: GPIO External Interrupt (EXTI)

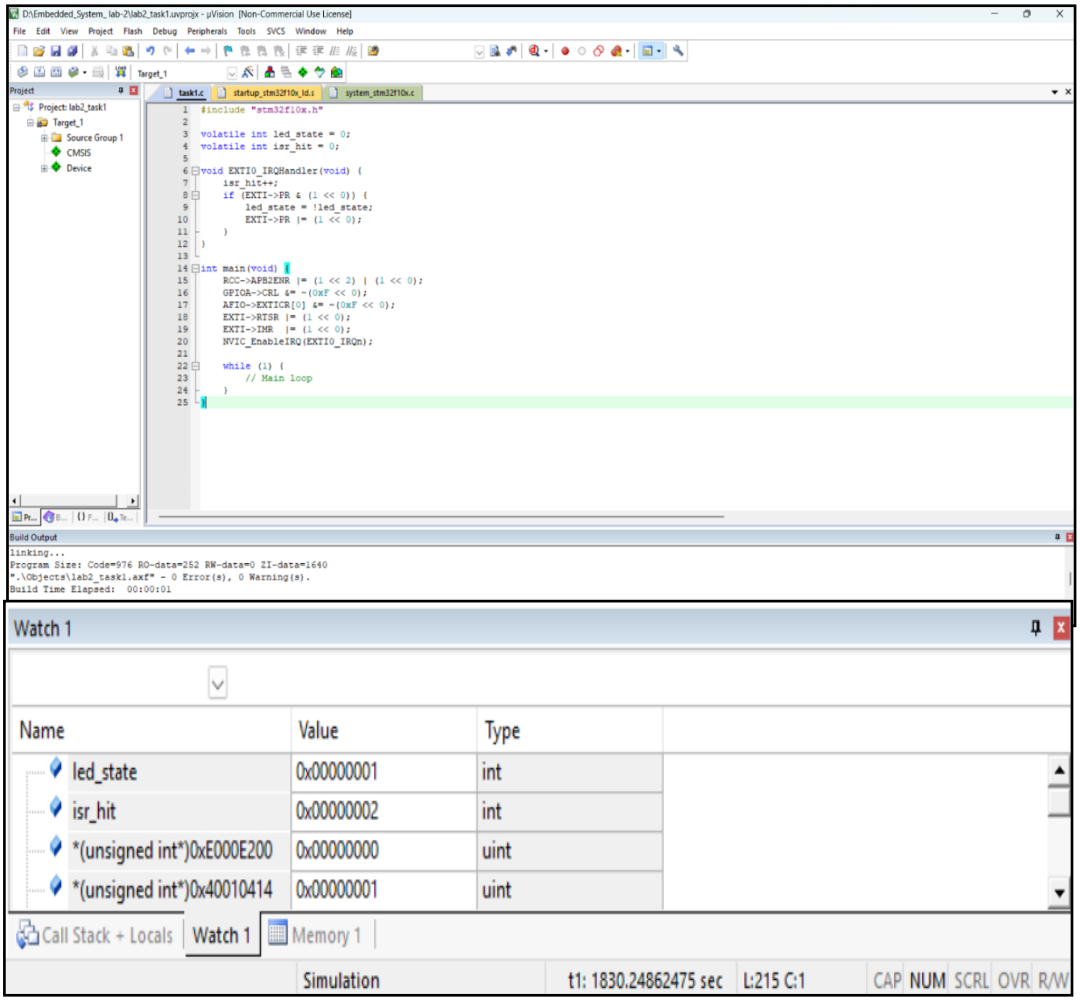
Objective

To understand how a GPIO pin event (such as a button press) generates an interrupt and how the ISR responds.

Description

In this task, **PA0** was configured as an input pin connected to **EXTI Line 0** with **rising-edge trigger** detection. An ISR named EXTI0_IRQHandler() was written to toggle a software variable led_state each time the interrupt is triggered. In debug mode, the signal change was simulated using the watch window and by manually writing to the EXTI->PR register. The change in led_state was observed in the **Expressions/Watch** window after each interrupt, confirming that the ISR executes correctly when a rising edge is detected on PA0.

Code and Output



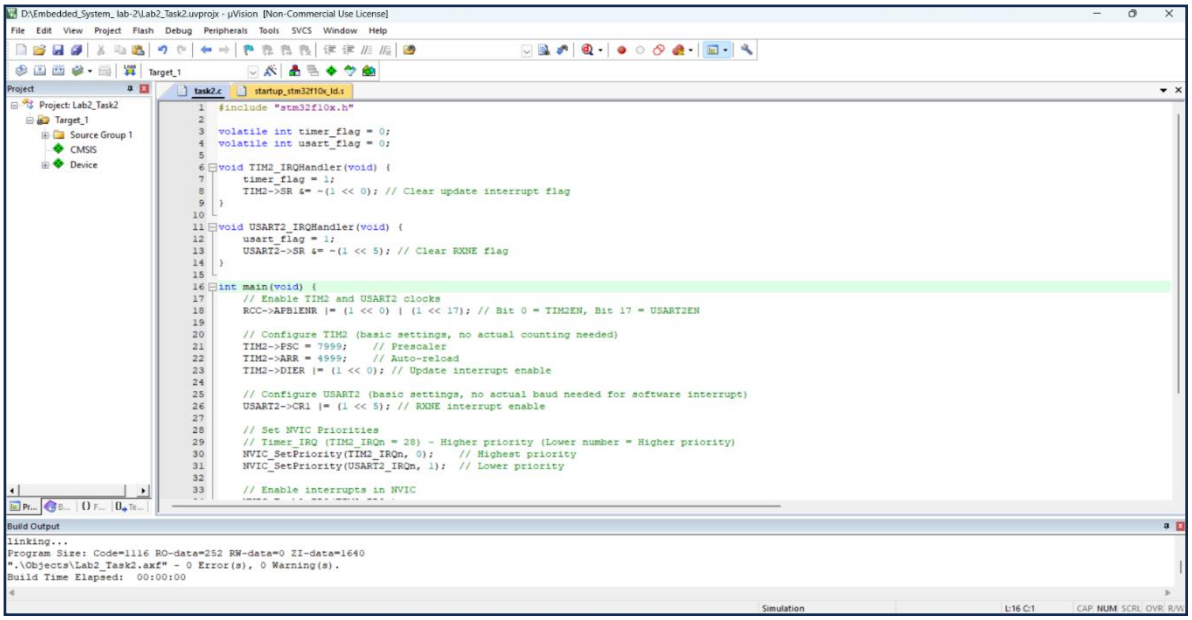
How I Learned
In this task, I learned how a **physical event on a GPIO pin** (such as a button press) can generate an interrupt. I configured **PA0** as an input with **EXTI Line 0** and set it to trigger on a **rising edge**. Writing the EXTI0_IRQHandler() function taught me that inside an ISR, I must first check the interrupt flag, perform the required action, and then **clear the pending bit** — otherwise the interrupt would fire repeatedly. By simulating the signal change in debug mode and watching the led_state variable toggle, I understood the complete flow: **event → interrupt request → NVIC → ISR → flag clear**.

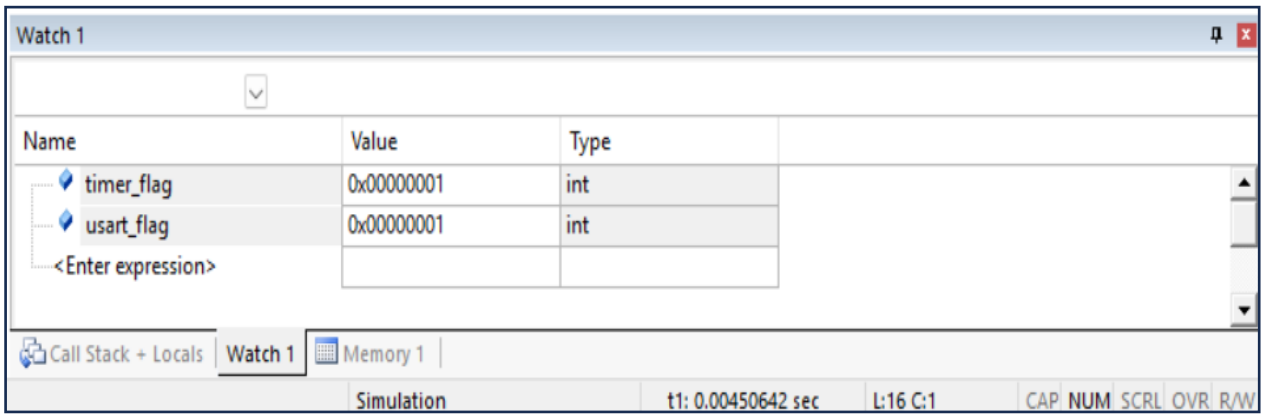
Task 2: Interrupt Priority & Preemption (NVIC)

Objective
To demonstrate interrupt priority and preemption using software-generated interrupts.

Description
Two dummy ISRs — Timer_IRQ_Handler() and USART_IRQ_Handler() — were created and assigned different priorities in the **NVIC**. The Timer interrupt was given a **higher priority** and the USART interrupt a **lower priority**. Both interrupts were triggered using software calls (NVIC_SetPendingIRQ()), and the execution order was observed in the **debug call stack**. The priorities were then swapped and the experiment repeated, clearly showing the effect of **preemption** — the higher-priority ISR preempts the lower-priority one.

Code and Output





How I Learned
This task taught me one of the most important concepts in embedded systems: **interrupt priority and preemption**. I created two dummy ISRs (Timer_IRQ_Handler and USART_IRQ_Handler) and assigned different priorities in the **NVIC**. By triggering both interrupts using NVIC_SetPendingIRQ(), I observed in the **debug call stack** that the higher-priority interrupt executed first. When I swapped the priorities and repeated the experiment, the execution order changed accordingly. This made it clear to me that if a higher-priority interrupt occurs while a lower-priority ISR is running, the higher-priority one **preempts** the lower-priority one.

Task 3: Timer Interrupt — Periodic LED Blinking

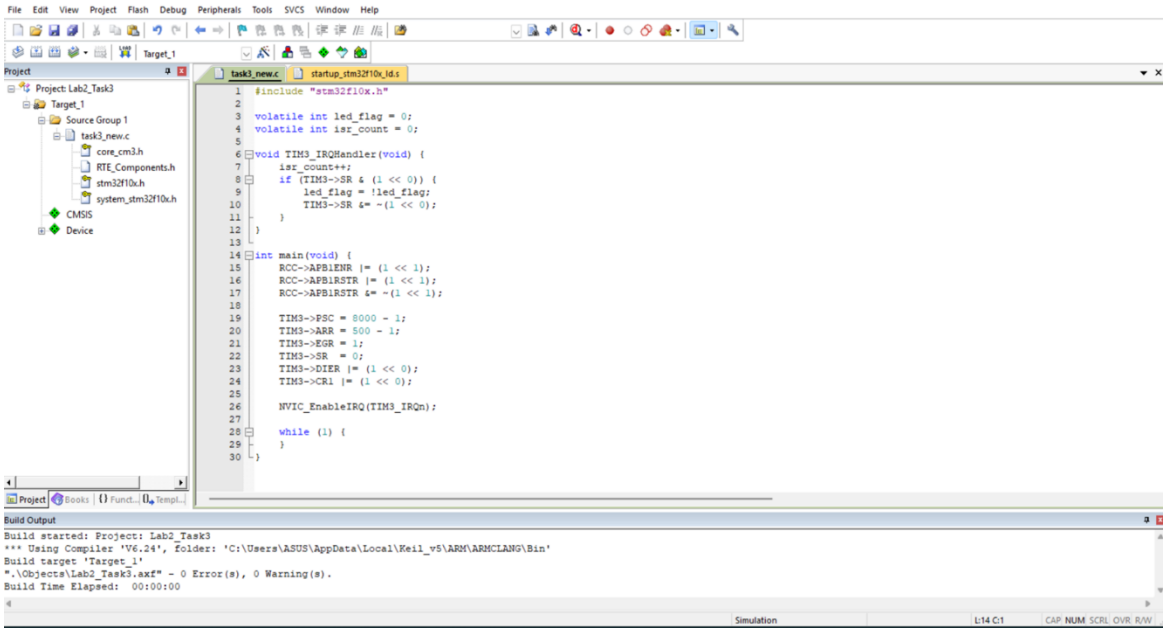
Objective
To generate periodic interrupts using a timer and simulate LED blinking behavior.

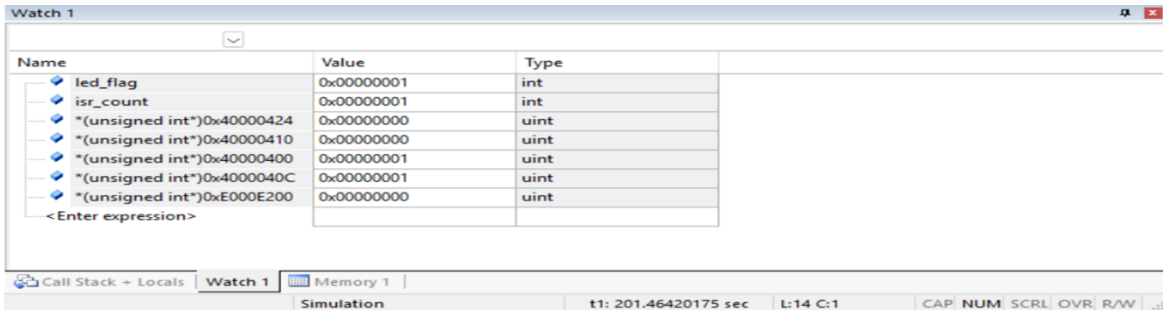
Description
Timer2 was configured to generate an interrupt every **500 ms** using an appropriate prescaler and auto-reload value. In the ISR TIM2_IRQHandler(), a software variable led_flag was toggled on every update event. The simulation was run and the value of led_flag was observed in the **Expressions** window, where it changed state every 500 ms. The prescaler and auto-reload values were then adjusted to change the blinking speed, verifying the relationship between timer configuration and interrupt frequency.

Timer Calculation

- System clock = 72 MHz
- Prescaler = 7200 − 1 → Timer clock = 10 kHz
- Auto-reload = 5000 − 1 → Interrupt every 5000 / 10 kHz = **500 ms**

Code and Output





How I Learned

In this task, I learned how to use a **hardware timer (TIM2)** to generate periodic interrupts. I configured the **prescaler** and **auto-reload register (ARR)** so that the timer produced an interrupt every **500 ms**. Inside the ISR, I toggled a software variable called led_flag. Watching this variable in the **Expressions** window showed it changing state exactly every 500 ms, which proved that the timer was working correctly. When I changed the prescaler and ARR values, the blinking speed changed as expected — this helped me understand the relationship between timer clock, prescaler, ARR, and interrupt frequency.

Task 4: UART Interrupt Handling (Virtual Console)

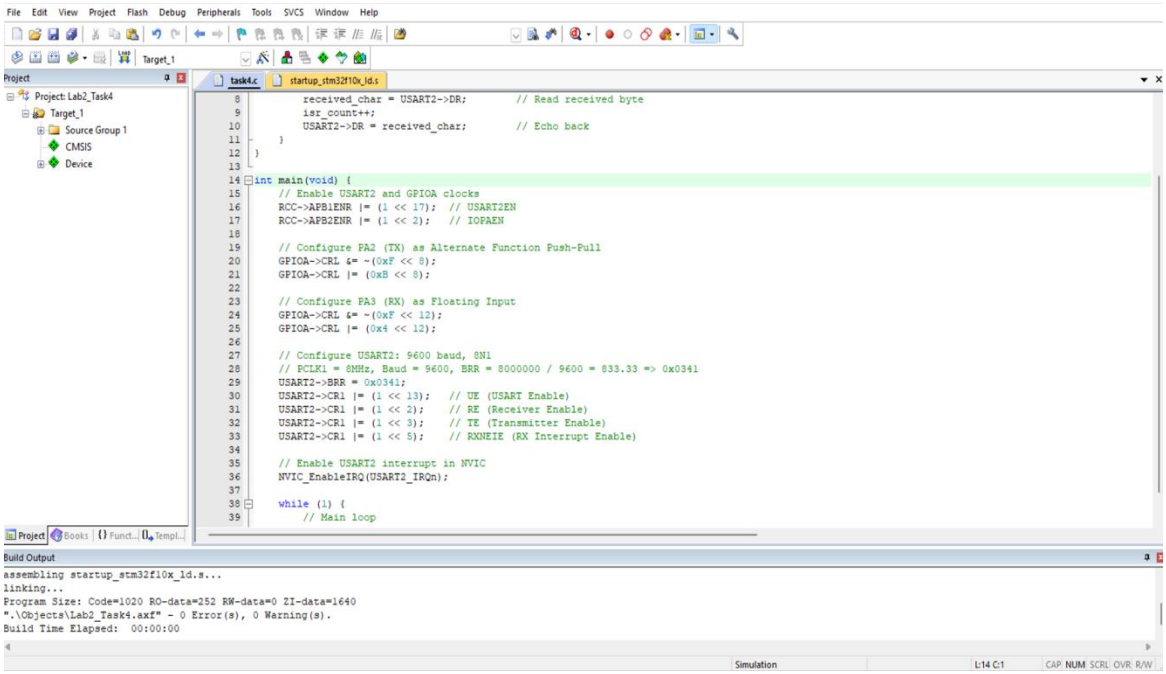
Objective

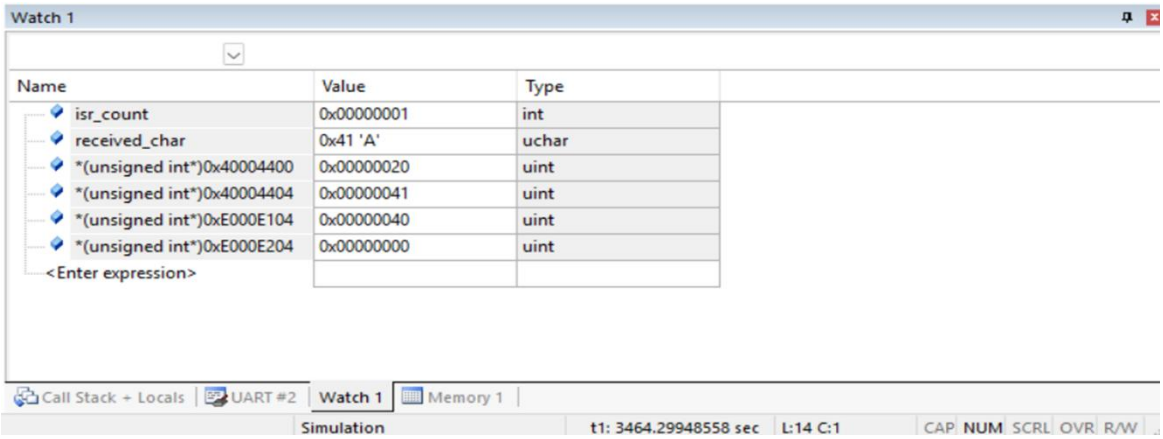
To simulate UART-based interrupt handling using a virtual console.

Description

USART2 was initialized for serial communication at 9600 baud, and the **RX** interrupt **(USART_IT_RXNE)** was enabled. Inside the **ISR USART2_IRQHandler()**, the received byte was read from the data register and echoed back using **HAL_UART_Transmit()**. Characters were sent through the simulation console, and each received byte triggered the ISR, which echoed the character back to the console. This demonstrated how **UART** interrupt-driven communication works — the CPU does not poll for incoming data but is notified by the interrupt.

Code and Output





How I Learned

This task showed me how **UART communication** can be handled using interrupts instead of polling. I initialized **USART2** at **9600 baud** and enabled the **RX interrupt** (USART_IT_RXNE). Inside the ISR USART2_IRQHandler(), I read the received byte from the data register and echoed it back using HAL_UART_Transmit(). By sending characters through the **simulation console**, I observed that each received byte triggered the ISR, and the echoed character appeared back on the console. This taught me that interrupt-driven UART is much more efficient than polling, because the CPU is only interrupted when data actually arrives.

Task 5: Fault Exceptions (HardFault, UsageFault, BusFault)

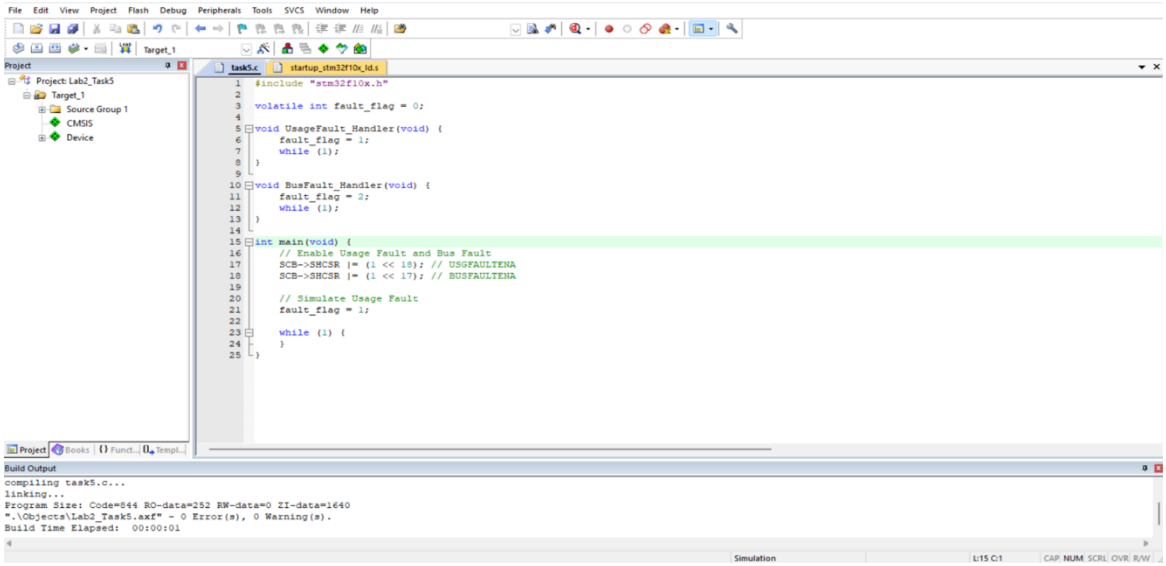
Objective

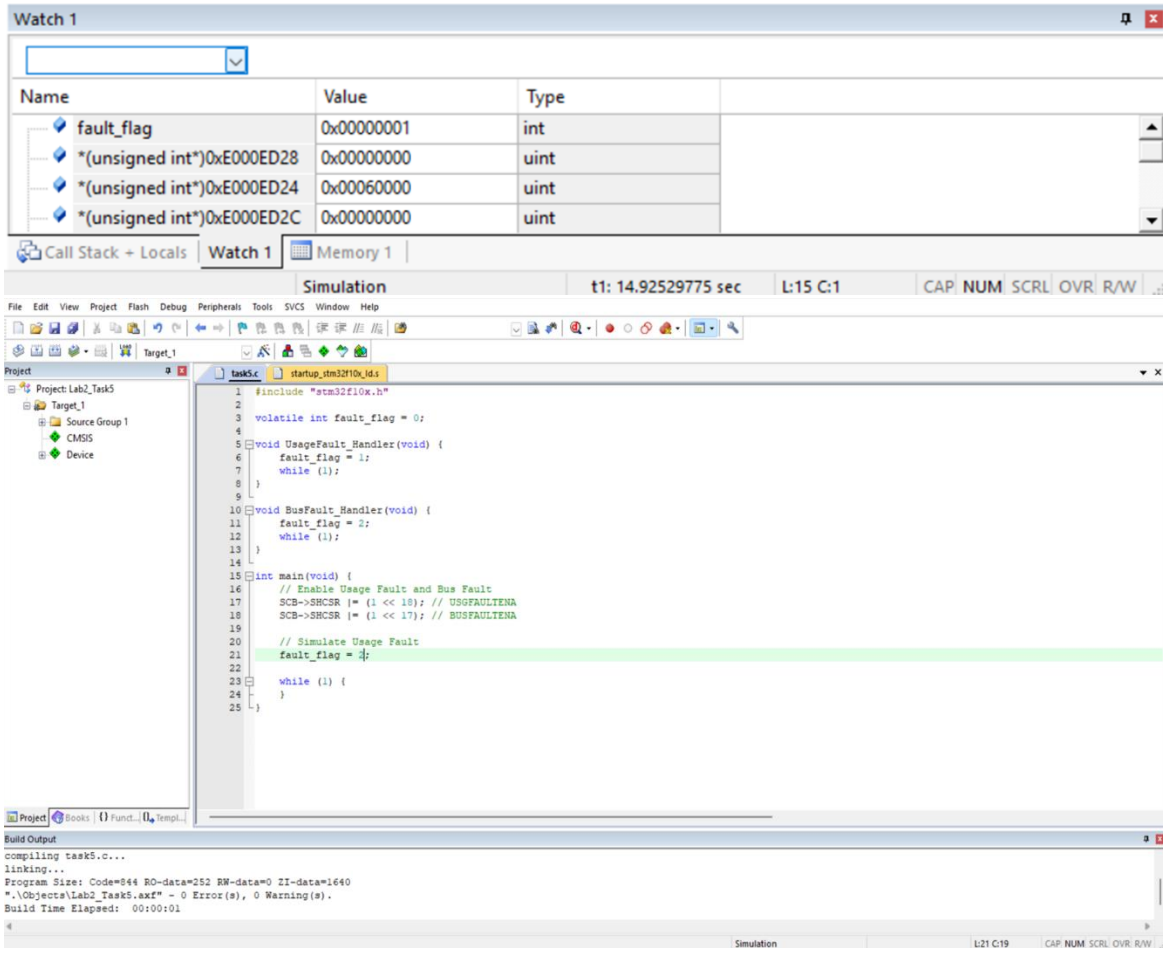
To observe how Cortex-M3 handles fault exceptions such as HardFault and UsageFault.

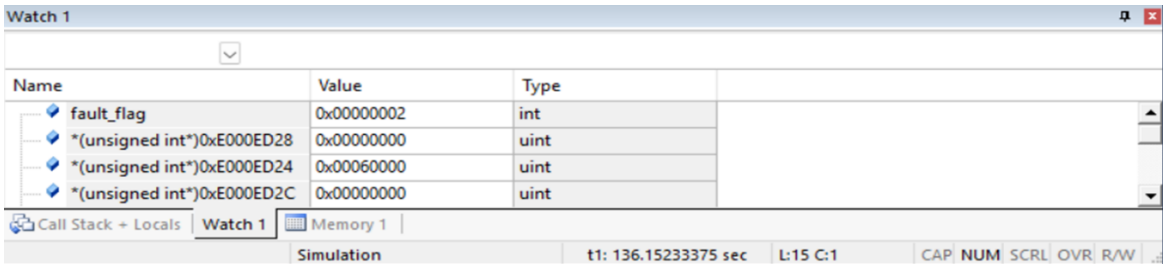
Description

Two fault conditions were tested: a **Usage Fault** caused by integer division by zero (int x = 1 / 0;), and a **Bus Fault** caused by writing to an invalid memory address (*(volatile int*)0xFFFFFFFF = 5;). The corresponding fault handlers (HardFault_Handler, UsageFault_Handler, BusFault_Handler) were enabled in the **SHCSR** register. Breakpoints were placed inside each fault handler, and when the fault occurred, execution stopped inside the appropriate handler. The **stack frame** was then inspected in the debugger to understand how Cortex-M3 saves context before entering a fault exception.

Code and Output







How I Learned

This task introduced me to the **fault exception mechanism** of the Cortex-M3. I deliberately created two fault conditions: a **Usage Fault** by dividing by zero (int x = 1 / 0;) and a **Bus Fault** by writing to an invalid memory address (*(volatile int*)0xFFFFFFFF = 5;). I enabled the fault handlers in the **SHCSR** register and placed breakpoints inside HardFault_Handler, UsageFault_Handler, and BusFault_Handler. When the fault occurred, execution stopped inside the correct handler, and I was able to inspect the **stack frame** to see how Cortex-M3 saves the context (registers, return address, etc.) before entering the fault exception. This was a very valuable lesson in **debugging real-world embedded faults**.

Conclution

Overall, this lab gave me a solid foundation in the Cortex-M3 exception and interrupt system. I learned that interrupts are not just a programming technique — they are a fundamental mechanism that allows embedded systems to respond to the real world in real time. From configuring a simple GPIO interrupt to debugging complex fault exceptions, every task added a new layer of understanding. The skills I gained — writing ISRs, setting NVIC priorities, configuring timers and UART interrupts, and debugging fault exceptions — will be directly applicable when I work with real STM32 hardware in the future. Most importantly, I now feel confident in applying interrupt-driven design rather than relying on inefficient polling loops.